

EduTrack

College Management System

Full Technical Documentation v1.0

What is EduTrack?

EduTrack is a full-stack college management web application built with Python (Flask) on the backend and vanilla HTML/CSS/JavaScript on the frontend. It provides a centralised portal for four distinct user roles -- Admin, Teacher, Student, and Parent -- each with a personalised dashboard, live notifications, and full database persistence using SQLite (dev) or PostgreSQL (prod).

Key Highlights:

- Role-based login with Portal IDs, OTP email reset, and biometric stub
- GPS-based classroom geofencing with live session check-in
- Real-time in-app messaging & notification system (polling every 30s)
- Full CRUD for Teachers, Students, and Parents with auto-UID generation
- Finance module: fee structures, payments, salary disbursements
- Marks entry & auto-notification to students and parents
- Docker-ready with Gunicorn production server support

Tech Stack:	Python 3.11 * Flask 3 * SQLAlchemy * SQLite / PostgreSQL * Vanilla JS
Frontend:	Single-Page Application (SPA) * Lucide Icons * Chart.js * CSS Custom Properties
Auth:	Session-based * Portal UID / Email login * OTP via EmailJS * Biometric stub
Deployment:	Gunicorn + Docker * GitHub repository

Table of Contents

1. Project Structure & File Map
2. How The App Starts (Application Factory)
3. Database: Models & Tables
4. Authentication & User Roles
5. API Blueprint Reference
6. Directory & UID Auto-Generation
7. Attendance System (Manual + GPS Session)
8. Finance Module
9. Marks & Notifications
10. In-App Messaging System

1. Project Structure & File Map

EduTrack follows the Flask Application Factory pattern. All Python application code lives inside the `app/` package, while static assets and the single-page HTML shell live at the project root / static folder.

```
Edutrack/
  app/
    __init__.py      # Flask application package
    models.py        # Application Factory -- create_app()
    models.py        # All SQLAlchemy DB models
    auth.py          # Login, logout, OTP, biometric endpoints
    directory.py      # Teacher / Student / Parent CRUD
    attendance.py     # Manual attendance + GPS live session
    marks.py         # Grade entry and retrieval
    finance.py        # Fee structures, payments, salaries
    notifications.py  # Alerts + in-app messaging API
    curriculum.py     # Curriculum file upload/download
    reports.py        # Admin stats & report aggregations
    profile.py        # User profile management
    config.py         # Config classes (Dev / Prod)
    face_recognition.py # GPS Haversine + face stub helpers
  static/
    css/style.css     # All app styling (2500+ lines)
    js/main.js        # Full SPA logic (~2700 lines)
    js/sw.js          # Service Worker (PWA offline cache)
    manifest.json     # PWA manifest
  index.html          # Single HTML shell (all views in DOM)
  wsgi.py             # Entry point for Gunicorn / python run
  requirements.txt    # Python dependencies
  Dockerfile          # Docker build instructions
  docker-compose.yml  # Multi-service docker setup
  .env               # Secret keys (not committed to git)
```

How Requests Flow

Browser -> index.html (loaded once) -> JavaScript makes `fetch()` API calls -> Flask blueprint routes -> SQLAlchemy ORM -> SQLite database -> JSON response -> JavaScript updates the DOM without page refresh.

2. How The App Starts (Application Factory Pattern)

The entire Flask application is created inside the `create_app()` function in `app/__init__.py`. This pattern avoids circular imports and allows the same codebase to run in dev, production, or test modes simply by changing the `FLASK_ENV` environment variable.

Step-by-Step Startup Sequence

- `FLASK_ENV` is read -> selects Dev or Prod config (`config.py`)
- `Flask()` object is created with the static folder path
- `SQLAlchemy db.init_app(app)` is called -- connects ORM to the app
- `Flask-Migrate migrate.init_app(app, db)` is called -- handles schema versioning
- All Blueprints are imported and registered with URL prefixes (e.g., `/api/auth`)
- Within app context: `db.create_all()` creates tables if they do not exist
- Auto-migration block runs: `ALTER TABLE` for new columns on existing DBs
- `seed_database(app)` is called -- inserts demo data on first run only
- The `/` route is registered to serve `index.html`

Registered Blueprints & Their URL Prefixes

Blueprint	File	URL Prefix	Purpose
<code>auth_bp</code>	<code>auth.py</code>	<code>/api/auth</code>	Login, logout, OTP, biometric
<code>attendance_bp</code>	<code>attendance.py</code>	<code>/api/attendance</code>	Attendance management
<code>directory_bp</code>	<code>directory.py</code>	<code>/api/directory</code>	People CRUD
<code>finance_bp</code>	<code>finance.py</code>	<code>/api/finance</code>	Fees & salaries
<code>marks_bp</code>	<code>marks.py</code>	<code>/api/marks</code>	Grades
<code>notifications_bp</code>	<code>notifications.py</code>	<code>/api/notifications</code>	Alerts & messages
<code>curriculum_bp</code>	<code>curriculum.py</code>	<code>/api/curriculum</code>	File uploads
<code>reports_bp</code>	<code>reports.py</code>	<code>/api/reports</code>	Admin analytics
<code>profile_bp</code>	<code>profile.py</code>	<code>/api/profile</code>	User profile

Auto-Migration System

Because the database is SQLite in development and may already have data, a custom auto-migration block uses SQLAlchemy's `inspect()` to check existing columns and runs raw `ALTER TABLE SQL` if a new column is missing. This means you never lose data when a new feature adds a column.

Seed Data (First-Run Only)

If the `users` table is empty, `seed_database()` inserts 9 demo profiles -- 1 admin, 3 teachers, 2 students, 2 parents, plus campus settings, fee structures, a sample mark, and a salary disbursement. All demo passwords are `demo123`.

3. Database: Models & Tables

All database tables are defined using SQLAlchemy ORM in app/models.py. SQLite is used automatically in development. For production, set the DATABASE_URL environment variable to a PostgreSQL connection string.

Table Name	Description
users	Central login table. Holds email, password hash, role, and foreign keys to teacher/student/parent rows. Also stores Portal UID, OTP fields.
teachers	Teacher profile: name, email, department, monthly salary, phone numbers, DOB, blood group, profile picture.
students	Student profile: roll number, name, email, department, parent link, contact fields, placement flag.
parents	Parent profile: name, email, phone, contact fields.
notifications	One notification row per user per event. Has title, message, type (info/success/warning/danger), and is_read flag.
messages	In-app messages. Sender -> recipient (individual, role group, or department). Links to notifications for bell badge.
attendance_records	One record per student per course per date. Includes method: manual, gps_face, gps_session, biometric, qr.
class_sessions	Live GPS classroom session started by a teacher. Stores lat/lng/radius and lifecycle timestamps.
session_checkins	Each GPS ping or initial check-in during a live session. Stores distance and inside_radius flag.
marks	Grade entry: student, teacher, course code, exam title, score, max_score.
fee_structures	Programme fee items (tuition, lab etc.) per academic year.
fee_payments	Payment record: which student paid, which structure, amount, date.
salary_disbursements	Monthly teacher salary records: gross, deductions, net.
campus_settings	Single row (id=1): campus GPS coordinates, radius, departments JSON list.
academic_events	Calendar events: title, date, type (calendar / event).
exam_schedules	Upcoming exam rows: course code, title, date.

Key Relationships

- users.teacher_id -> teachers.id (FK)
- users.student_id -> students.id (FK)
- users.parent_id -> parents.id (FK)
- students.parent_id -> parents.id (FK, optional)
- marks: student_id -> students.id, teacher_id -> teachers.id
- attendance_records: student_id -> students.id
- messages: sender_id -> users.id, recipient_id -> users.id (nullable for broadcast)

4. Authentication & User Roles

Login Methods

- Password Login: POST /api/auth/login with identifier (Portal UID or email) + password. Uses werkzeug.security.check_password_hash. Sets Flask session["user_id"].
- OTP Login: POST /api/auth/request-otp -> generates 6-digit OTP, stores hash+expiry on the User row, sends via EmailJS API. POST /api/auth/verify-otp-login to log in.
- Biometric Login (Stub): POST /api/auth/biometric/login with identifier. In demo mode this always succeeds. Real WebAuthn would replace the stub.
- Password Reset: request-otp -> verify OTP -> POST /api/auth/reset-password with new password.

Four User Roles

ADMIN	Full system access. Can view all users, manage fees, salaries, campus GPS, OTP logs, departments, and send messages to anyone.
TEACHER	Can manage attendance for their classes, grade students, view salary slips, and send messages to students/parents.
STUDENT	Can view own marks, attendance %, curriculum files, and check in to live GPS sessions.
PARENT	Can view child's fee payment records and receive notifications about attendance/marks.

Session Management

Flask's server-side session stores the user_id. The require_login() helper function (in auth.py) is called at the top of every protected route. It reads session["user_id"], looks up the User row, and returns the user object or a 401 JSON error. The JavaScript frontend always calls /api/auth/me on page load to restore state.

Portal UID Format

Role	Pattern	Example
Teacher	EMP-{DEPT}{YY}{NAME3}{SEQ}	EMP-CSE26SAR001
Student	STU-{DEPT}{YY}{NAME3}{SEQ}	STU-CSE26ALE001
Parent	PAR-{YY}{NAME3}{SEQ}	PAR-26MRS001

UIDs are generated in directory.py using regex helpers. The sequence number is determined by counting how many existing UIDs share the same prefix, then zero-padding to 3 digits. This guarantees uniqueness without a separate counter.

5. API Blueprint Reference -- All Endpoints

AUTH /api/auth		
POST	/login	Login with identifier + password
POST	/logout	Clear session
GET	/me	Get current logged-in user
POST	/request-otp	Send OTP to email
POST	/verify-otp-login	Login via OTP code
POST	/reset-password	Reset password after OTP verify
POST	/biometric/challenge	Get WebAuthn challenge
POST	/biometric/verify	Complete biometric auth
POST	/biometric/login	Passwordless biometric login
GET	/admin/otp-logs	View active OTPs (admin only)
GET	/admin/departments	Get department list
POST	/admin/departments	Update department list
DIRECTORY /api/directory		
GET	/teachers	List all teachers
POST	/teachers	Add new teacher + auto-create user
PATCH	/teachers/<id>	Edit teacher full profile
DELETE	/teachers/<id>	Delete teacher + cascade user
GET	/students	List all students
POST	/students	Add new student
PATCH	/students/<id>	Edit student full profile
DELETE	/students/<id>	Delete student
GET	/parents	List all parents
POST	/parents	Add new parent
PATCH	/parents/<id>	Edit parent profile
DELETE	/parents/<id>	Delete parent
ATTENDANCE /api/attendance		
GET	/campus	Get campus GPS settings
PUT	/campus	Update campus GPS (admin)
POST	/verify-location	Check if coordinates are inside campus
GET	/manual	Get attendance roster for course+date
POST	/manual	Save bulk attendance entries
POST	/mark-combined	Student self-mark (GPS + face)
POST	/session/start	Teacher starts a live GPS session
POST	/session/end	Teacher ends the live session
GET	/session/active	Get currently active session(s)
POST	/session/checkin	Student checks in to live session
POST	/session/ping	Background GPS ping from student
NOTIFICATIONS /api/notifications		
GET	/list	Get all notifications for current user
GET	/unread-count	Get count of unread notifications (badge polling)
POST	/mark-read	Mark one or all notifications as read

POST	/messages/send	Send a message (admin/teacher only)
GET	/messages/inbox	Get received messages
GET	/messages/sent	Get sent messages
POST	/messages/mark-read/<id>	Mark a message as read
GET	/messages/recipients	Get list of sendable users

FINANCE	/api/finance
----------------	--------------

GET	/fee-structures	List fee structures
POST	/fee-structures	Add fee structure (admin)
GET	/fee-payments	List payments (role-filtered)
POST	/fee-payments	Record a payment (admin)
GET	/salaries	List salary records
POST	/salaries	Disburse salary (admin)

MARKS	/api/marks
--------------	------------

GET	/list	Get marks (role-filtered)
POST	/add	Add grade + notify student/parent

7. Attendance System -- Deep Dive

A. Manual Attendance (Teacher)

Teachers open the Attendance tab, pick a course code and date, and see a roster of all students. They toggle each student Present/Absent and click Save. The frontend sends a POST `/api/attendance/manual` with an array of entries.

If a student is marked absent, the system automatically creates two Notification rows -- one for the student, one for their linked parent -- with type "danger".

B. GPS + Face Recognition Self-Marking (Student)

- Student clicks "Mark Attendance" -> browser requests GPS coordinates.
- POST `/api/attendance/mark-combined` sends lat/lng + selfie image.
- Server computes Haversine distance between student and campus GPS point.
- If distance > campus radius -> rejected with "Outside campus boundary" error.
- If inside radius -> face image is checked by `verify_face_stub()` (demo: always passes).
- On success -> AttendanceRecord is upserted. Student gets a "Attendance Marked" notification.

C. Live GPS Classroom Session (Teacher + Students)

This is the most advanced attendance feature. Here is the full flow:

1. Teacher opens their GPS session panel and clicks "Start Session".
2. Browser gets teacher GPS -> POST `/api/attendance/session/start`.
3. Server creates a ClassSession row with teacher lat/lng/radius (default 20m).
4. All students receive a "Class Started" notification immediately.
5. Student opens app, sees session banner -> clicks "Check In".
6. Browser gets student GPS + captures selfie -> POST `/api/attendance/session/checkin`.
7. Server checks: is student within session radius? Is face present? If yes -> SessionCheckIn row (type=initial).
8. After check-in, JS starts background random pings every 2-5 minutes.
9. Each ping sends GPS only (no photo) to POST `/api/attendance/session/ping`.
10. If student is found outside the radius during a ping -> teacher gets a "Student Left" warning notification.
11. Teacher clicks "End Session" -> POST `/api/attendance/session/end`.
12. Server counts unique students with `inside_radius=True` check-ins -> saves to AttendanceRecord.

Haversine Distance Formula

The `haversine_m()` function in `face_recognition.py` computes great-circle distance between two GPS coordinates (lat/lng) in metres. This is the standard formula used for GPS boundary checks -- it accounts for Earth's curvature correctly over short distances.

10. In-App Messaging & Notification System

Notification Bell (All Users)

Every user has a notification bell in the top navigation bar. Every 30 seconds, the JavaScript polls GET /api/notifications/unread-count. If count > 0, the badge shows the number and pulses with a CSS animation. Clicking the bell opens a glassmorphism dropdown panel with three tabs.

Tab	Who Sees It	Contents
Alerts	Everyone	System notifications (absent alert, new marks, class started, etc.)
Inbox	Everyone	Direct messages and broadcast messages received
Sent	Admin / Teacher	All messages sent by this user

Sending Messages (Admin + Teacher Only)

A "Compose" button appears in the panel footer for admin and teacher users. It opens a modal with three targeting modes:

- Individual -- pick a specific user from a dropdown (role-labeled)
- Role Group -- message all students, all teachers, all parents, or everyone
- Department -- message all students + teachers in a specific department

When a message is sent, the backend fans out: it creates a Notification row for every recipient. This means the bell badge lights up for each recipient within the next 30-second poll cycle.

Teacher Restrictions

Teachers can only message students and parents. They cannot message other teachers or administrators. This restriction is enforced on the backend in /api/notifications/messages/send and on the frontend recipient dropdown.

Auto-Generated Notifications

The following actions automatically create notification rows in the database:

Action	Who Gets Notified	Type
Teacher marks student absent	Student + Parent	danger (red)
Teacher adds marks	Student + Parent	success / info
Teacher starts GPS session	All students	info
Student checks in successfully	Student only	success
Student pings outside classroom	Teacher	warning
Admin/Teacher sends a message	All recipients	info

11. Frontend Architecture -- Single-Page Application

How It Works

EduTrack is a Single-Page Application (SPA). The browser loads index.html exactly once. All views -- login screen, dashboards, modals -- are already in the HTML DOM, just hidden with CSS class "hidden". JavaScript toggles visibility to simulate navigation without page reloads.

main.js Structure (inside an IIFE)

```
(function() {
  "use strict";

  // Constants
  const NAV = { admin: [...], teacher: [...], student: [...], parent: [...] };
  const DASH_MAP = { admin: "adminDashboard", teacher: "teacherDashboard", ... };

  // State
  let state = { user: null, apiOnline: false, role: null };

  // Core helpers: apiFetch(), apiJson(), tryApi(), escapeHtml()
  // Chart helpers: initAdminCharts(), destroyCharts()
  // Render functions: renderStudentTableFromApi(), loadDirectoryAdmin(), ...
  // Section loaders: loadFeesAdmin(), loadSalaryAdmin(), ...
  // Attendance: startClassSession(), endClassSession(), checkActiveSessionStudent()
  // Navigation: buildNav(), showDashboard(), hideDashboard()
  // Event listeners: loginForm.submit, logoutBtn.click, ...

  // App init: DOMContentLoaded -> tryApi() -> showDashboard(role, user)
})();
```

Key JavaScript Functions

Function	Purpose
apiJson(path, opts)	Fetch JSON from backend; throws on non-2xx responses
showDashboard(role, user)	Show role dashboard; hide login; start notification polling
buildNav(role)	Build sidebar & bottom nav from NAV[role] config array
loadDirectoryAdmin()	Fetch & render Teachers, Students, Parents tables
openEditProfileModal(type, data)	Show edit modal pre-filled with profile data
startClassSession()	Get GPS, POST to /session/start, begin polling
startRandomPings(sessionId)	Background GPS ping every 2-5 minutes
initNotificationPanel()	Wire bell button, poll unread count every 30s
openComposeModal()	Load recipient list, open compose message modal
_showToast(msg)	Display bottom-right toast notification for 3 seconds

Navigation System

Navigation items are defined as JS objects {id, label, icon} per role in the NAV constant. buildNav() generates both a sidebar nav (desktop) and a bottom tab bar (mobile). Clicking a nav item adds class "active" to that button and shows/hides the matching .app-section div by its id.

CSS Architecture

All styles live in `static/css/style.css` (2500+ lines). The design system is defined as CSS Custom Properties (`--primary-600`, `--gray-200`, etc.) in the `:root` block. These are referenced throughout all components. The colour palette is Steel Blue + Warm Champagne with semantic Emerald, Coral, and Amber.

12. Deployment & Running Locally

Running Locally (Development)

```
# 1. Clone the repository
git clone https://github.com/hrishav2208/Edutrack.git
cd Edutrack

# 2. Create virtual environment
python -m venv venv
venv\Scripts\activate          # Windows
source venv/bin/activate       # macOS / Linux

# 3. Install dependencies
pip install -r requirements.txt

# 4. Set environment variables (optional for dev)
copy .env.example .env         # then edit .env

# 5. Run the server
python wsgi.py
# OR use the included batch file on Windows:
run.bat

# App runs at: http://127.0.0.1:5000
```

Environment Variables (.env)

Variable	Purpose	Example Value
SECRET_KEY	Flask session encryption key	any-random-secret-string
DATABASE_URL	Postgres URL for production	postgresql://user:pw@host/db
FLASK_ENV	Config profile	dev OR prod
EMAILJS_SERVICE_ID	EmailJS service for OTP	service_abc123
EMAILJS_TEMPLATE_ID	EmailJS template for OTP	template_xyz
EMAILJS_PUBLIC_KEY	EmailJS public key	user_pubkey
DEFAULT_CAMPUS_LAT	Default campus latitude	28.6139
DEFAULT_CAMPUS_LNG	Default campus longitude	77.2090
DEFAULT_CAMPUS_RADIUS_M	Geofence radius in metres	500

Production with Gunicorn

```
# Direct Gunicorn (Linux/Mac server)
gunicorn wsgi:app --workers=4 --bind=0.0.0.0:5000

# Or use the config file
gunicorn -c gunicorn.conf.py wsgi:app
```

Docker Deployment

```
# Build and start with Docker Compose
docker-compose up --build
```

```
# The Dockerfile:
#   Base: python:3.11-slim
#   Installs requirements.txt
#   Runs: gunicorn wsgi:app
#   Exposes port 5000
```

Default Demo Login Credentials

Role	Email / UID	Password
Admin	admin@edutrack.com	demo123
Teacher (Dr. Sarah Chen)	teacher@edutrack.com	demo123
Teacher (Hrishav Bisht)	EMP-AI26HRI001	demo123
Student (Alex Kumar)	student@edutrack.com	demo123
Student (Wilson Gaikwad)	STU-AIM26WIL001	demo123
Parent (Mrs. Sharma)	parent@edutrack.com	demo123
Parent (Sanjay Gaikwad)	PAR-26SAN001	demo123

Quick Summary -- The 3 Things That Make EduTrack Work

- 1. Flask Application Factory (app/__init__.py) boots everything: config, DB, blueprints, seed data.
- 2. SQLAlchemy ORM (app/models.py) defines all 16 tables; db.create_all() + auto-migration keeps schema fresh.
- 3. JavaScript SPA (static/js/main.js) is a ~2700-line IIFE that handles all UI state, API calls, and dynamic rendering.